# TP5 — Portefeuilles cryptographiques et signatures numériques avec Python

**Matière :** Blockchain
**Environnement :** Python 3, VS Code, PyCharm ou Jupyter Notebook
**Bibliothèques :** hashlib, json et cryptography

## Objectifs pédagogiques

À la fin de ce TP, vous  serez capables de :

- Comprendre le rôle d'un portefeuille cryptographique dans une blockchain.
- Distinguer une clé publique d'une clé privée.
- Expliquer le principe de la cryptographie asymétrique.
- Comprendre le fonctionnement d'une signature numérique.
- Générer une paire de clés cryptographiques avec Python.
- Signer une transaction à l'aide d'une clé privée.
- Vérifier l'authenticité et l'intégrité d'une transaction.
- Comprendre comment les signatures numériques contribuent à la sécurité d'une blockchain.

# Partie I — Rappel-cours : cryptographie et sécurité des transactions

## 1. Rappel : les transactions dans une blockchain

Dans le TP4, nous avons développé un programme permettant de représenter et de valider des transactions.

Une transaction contenait trois informations principales :

- L'expéditeur.
- Le destinataire.
- Le montant.

Exemple :

Alice envoie 20 unités à Bob.

Nous avons également implémenté une fonction permettant de vérifier que l'expéditeur possède suffisamment de fonds.

Cependant, notre programme présente encore une faiblesse importante.

### Problème

Supposons qu'Alice possède 100 unités.

Bob pourrait créer une transaction contenant :

- Expéditeur : Alice.
- Destinataire : Bob.
- Montant : 80 unités.

Notre programme précédent pourrait accepter cette transaction si Alice possède suffisamment de fonds.

Mais comment savoir si Alice a réellement autorisé ce transfert ?

La vérification du solde ne suffit pas.

Nous devons disposer d'un mécanisme permettant de vérifier l'autorisation de l'expéditeur.

C'est le rôle des **signatures numériques**.

## 2. Qu'est-ce qu'un portefeuille cryptographique ?

Un portefeuille cryptographique, ou *wallet*, est un outil permettant à un utilisateur de gérer les clés cryptographiques utilisées pour contrôler ses actifs sur une blockchain.

Contrairement à une idée répandue, un portefeuille ne stocke généralement pas directement les cryptomonnaies.

Les informations relatives aux actifs sont enregistrées dans l'état de la blockchain.

Le portefeuille permet notamment de :

- Générer et gérer des clés cryptographiques.
- Identifier un utilisateur ou un compte.
- Préparer des transactions.
- Signer des transactions.
- Consulter les informations associées à ses actifs.

### Exemple

Alice possède un portefeuille cryptographique.

Ce portefeuille lui permet de créer une transaction destinée à Bob.

Avant de transmettre cette transaction au réseau, Alice utilise sa clé privée pour produire une signature numérique.

Les autres participants peuvent ensuite vérifier cette signature à l'aide de sa clé publique.

Ils peuvent ainsi vérifier que la transaction a été signée avec la clé privée correspondant à cette clé publique.

# 3. Comprendre la cryptographie asymétrique

La cryptographie asymétrique repose sur l'utilisation de deux clés mathématiquement liées :

- Une clé privée.
- Une clé publique.

Ces clés possèdent des rôles différents.

## 3.1. La clé privée

La clé privée est une information secrète qui doit rester sous le contrôle de son propriétaire.

Elle permet notamment de produire des signatures numériques.

Dans notre exemple, Alice utilise sa clé privée pour signer ses transactions.

Si une autre personne obtient cette clé, elle peut potentiellement signer des transactions à la place d'Alice.

La protection de la clé privée est donc essentielle.

## 3.2. La clé publique

La clé publique peut être communiquée aux autres participants.

Elle permet de vérifier les signatures produites avec la clé privée correspondante.

Contrairement à la clé privée, elle n'a pas besoin d'être gardée secrète.

## Exemple

Alice possède :

- Une clé privée : conservée secrètement.
- Une clé publique : accessible aux personnes qui souhaitent vérifier ses signatures.

Alice signe une transaction avec sa clé privée.

Bob ou un autre participant vérifie cette signature avec la clé publique d'Alice.

## Tableau comparatif

| Élément | Clé privée | Clé publique |
|---|---|---|
| Confidentialité | Secrète | Peut être partagée |
| Fonction principale dans ce TP | Signer | Vérifier |
| Propriétaire | Alice | Associée à Alice |
| Risque en cas de divulgation | Usurpation possible | Pas de divulgation secrète |
| Utilisation | Création de signatures | Vérification de signatures |

# 4. Qu'est-ce qu'une signature numérique ?

Une signature numérique est une valeur cryptographique calculée à partir d'un message et d'une clé privée.

Elle permet notamment de vérifier deux propriétés.

**Authenticité cryptographique**

La signature permet de vérifier que le message a été signé à l'aide de la clé privée correspondant à une clé publique donnée.

**Intégrité**

Si le message signé est modifié, la vérification de la signature doit échouer.

## Exemple

Alice souhaite envoyer la transaction suivante :

Alice → Bob : 20 unités.

Elle utilise sa clé privée pour signer cette transaction.

La transaction et sa signature peuvent ensuite être transmises au réseau.

Un participant récupère :

- Le contenu de la transaction.
- La signature numérique.
- La clé publique d'Alice.

Il utilise ces informations pour vérifier la signature.

Si la transaction a été modifiée après sa signature, la vérification échoue.

## Attention

Une signature valide ne prouve pas automatiquement que l'expéditeur possède suffisamment de fonds.

Elle ne garantit pas non plus que la clé publique appartient réellement à une personne appelée Alice.

Ces aspects nécessitent des règles supplémentaires de validation et d'association entre identité, compte et clé publique.

# 5. Comment fonctionne une signature numérique ?

Le fonctionnement peut être résumé en deux opérations.

## Opération 1 : signer

Alice prépare un message :

```
Alice envoie 20 unites a Bob
```

Elle utilise sa clé privée pour produire une signature numérique.

Cette signature dépend du message et de la clé utilisée.

### Opération 2 : vérifier

Bob récupère le message et sa signature.

Il utilise la clé publique d'Alice pour vérifier la signature.

Si la vérification réussit, il peut considérer que le message correspond à une signature valide pour cette clé publique.

### Exemple de modification

Message signé :

```
Alice envoie 20 unites a Bob
```

Message reçu après modification :

```
Alice envoie 200 unites a Bob
```

La signature initiale ne doit plus être valide pour le message modifié.

Ce mécanisme permet de détecter les modifications non autorisées des données signées.

## 6. L'algorithme Ed25519

Dans ce TP, nous utiliserons l'algorithme **Ed25519**.

Ed25519 est un algorithme de signature numérique reposant sur la cryptographie à courbes elliptiques.

Il permet :

- De générer une paire de clés.
- De signer un message.
- De vérifier une signature.

Nous utiliserons la bibliothèque Python `cryptography`.

Ed25519 est choisi ici pour sa simplicité d'utilisation et ses propriétés cryptographiques.

Il ne s'agit pas de l'algorithme de signature utilisé par Bitcoin, qui emploie notamment ECDSA et Schnorr sur la courbe secp256k1.

## 7. Installer la bibliothèque cryptography

Ouvrez un terminal dans votre environnement Python.

Exécutez :

```
python -m pip install cryptography
```

Pour vérifier l'installation :

```
import cryptography
```

```
print(cryptography.__version__)
```

La bibliothèque `hashlib`, utilisée dans les TP précédents, est déjà incluse dans Python.

# Partie II — TP

## Exercice 1 — Générer une paire de clés cryptographiques

### Étape 1 : préparation

Créez un dossier nommé :

`TP5_Blockchain`

Dans ce dossier, créez un fichier :

`exercice1.py`

### Étape 2 : générer les clés

Écrivez le programme suivant :

```
from cryptography.hazmat.primitives.asymmetric.ed25519 import (
    Ed25519PrivateKey
)

from cryptography.hazmat.primitives import serialization

# Generation de la cle privee
cle_privee = Ed25519PrivateKey.generate()

# Recuperation de la cle publique
cle_publique = cle_privee.public_key()

# Conversion de la cle publique en octets
cle_publique_bytes = cle_publique.public_bytes(
    encoding=serialization.Encoding.Raw,
    format=serialization.PublicFormat.Raw
)

print("Cle publique :", cle_publique_bytes.hex())
```

### Travail demandé

1. Exécutez le programme.
2. Observez la clé publique générée.
3. Exécutez plusieurs fois le programme.
4. Comparez les clés publiques obtenues.
5. Modifiez le programme pour afficher la longueur de la clé publique en octets.

### Questions

1. Pourquoi générons-nous deux clés différentes ?

2. Quel est le rôle de la clé privée ?
3. Quel est le rôle de la clé publique ?
4. Pourquoi la clé privée doit-elle rester secrète ?
5. Pourquoi les clés changent-elles lors de nouvelles générations ?

# Exercice 2 — Signer un message avec une clé privée

## Étape 1 : créer le fichier

Créez un fichier nommé :

`exercice2.py`

## Étape 2 : écrire le programme

```
from cryptography.hazmat.primitives.asymmetric.ed25519 import (
    Ed25519PrivateKey
)

# Generation des cles
cle_privee = Ed25519PrivateKey.generate()
cle_publique = cle_privee.public_key()

# Message a signer
message = "Alice envoie 20 unites a Bob"

# Conversion en octets
message_bytes = message.encode("utf-8")

# Signature du message
signature = cle_privee.sign(message_bytes)

print("Message :", message)
print("Signature :", signature.hex())
```

## Travail demandé

1. Exécutez le programme.
2. Observez la signature obtenue.
3. Modifiez le message en remplaçant 20 par 50.
4. Exécutez à nouveau le programme.
5. Comparez les signatures obtenues.
6. Modifiez le programme pour afficher la longueur de la signature en octets.

## Questions

1. Quelle clé est utilisée pour signer le message ?
2. Pourquoi convertissons-nous le message en octets ?
3. Quel est le rôle de la méthode `sign()` ?

4. Une signature numérique est-elle identique à un hash ?
5. Pourquoi la signature dépend-elle du message ?

# Exercice 3 — Vérifier une signature numérique

Nous allons maintenant utiliser la clé publique pour vérifier une signature.

## Étape 1 : créer le fichier

Créez un fichier nommé :

`exercice3.py`

## Étape 2 : écrire le programme

```
from cryptography.hazmat.primitives.asymmetric.ed25519 import (
    Ed25519PrivateKey
)

from cryptography.exceptions import InvalidSignature

# Generation des cles
cle_privee = Ed25519PrivateKey.generate()
cle_publique = cle_privee.public_key()

# Message
message = "Alice envoie 20 unites a Bob"

message_bytes = message.encode("utf-8")

# Signature
signature = cle_privee.sign(message_bytes)

# Verification
try:

    cle_publique.verify(signature, message_bytes)

    print("Signature valide")

except InvalidSignature:

    print("Signature invalide")
```

## Travail demandé

1. Exécutez le programme.
2. Observez le résultat de la vérification.
3. Remplacez le message utilisé pour la vérification par :

`Alice envoie 200 unites a Bob`

4. Exécutez à nouveau le programme.
5. Testez une modification du nom du destinataire.

6. Comparez les résultats.

### Questions

1. Quelle clé permet de vérifier une signature ?
2. Quel est le rôle de la méthode `verify()` ?
3. Pourquoi la vérification peut-elle échouer ?
4. Une signature reste-t-elle valide si le message est modifié ?
5. Comment ce mécanisme contribue-t-il à l'intégrité des transactions ?

# Exercice 4 — Créer une classe Portefeuille

Nous allons maintenant représenter un portefeuille cryptographique avec une classe Python.

Chaque portefeuille possédera :

- Une clé privée.
- Une clé publique.
- Une méthode permettant de signer un message.

### Étape 1 : créer le fichier

Créez un fichier nommé :

`exercice4.py`

### Étape 2 : définir la classe

```
from cryptography.hazmat.primitives.asymmetric.ed25519 import (
    Ed25519PrivateKey
)

from cryptography.hazmat.primitives import serialization


class Portefeuille:

    def __init__(self):

        self.cle_privee = Ed25519PrivateKey.generate()

        self.cle_publique = self.cle_privee.public_key()

    def obtenir_adresse(self):

        cle_bytes = self.cle_publique.public_bytes(
            encoding=serialization.Encoding.Raw,
            format=serialization.PublicFormat.Raw
        )

        return cle_bytes.hex()
```

```
    def signer(self, message):

        return self.cle_privee.sign(
            message.encode("utf-8")
        )
```

Dans ce TP, l'adresse du portefeuille sera représentée directement par la clé publique encodée en hexadécimal.

Dans une véritable blockchain, une adresse peut être dérivée de la clé publique à l'aide de règles spécifiques au protocole.

## Étape 3 : créer deux portefeuilles

Ajoutez :

```
alice = Portefeuille()
bob = Portefeuille()

print("Adresse Alice :", alice.obtenir_adresse())
print("Adresse Bob :", bob.obtenir_adresse())
```

## Étape 4 : signer un message

Ajoutez :

```
message = "Alice envoie 20 unites a Bob"

signature = alice.signer(message)

print("Signature :", signature.hex())
```

## Travail demandé

1. Exécutez le programme.
2. Créez un troisième portefeuille pour Charlie.
3. Affichez les adresses des trois portefeuilles.
4. Faites signer un message par Alice.
5. Faites signer un autre message par Bob.
6. Comparez les signatures.

## Questions

1. Quel est le rôle de la classe `Portefeuille` ?
2. Pourquoi chaque portefeuille possède-t-il sa propre paire de clés ?
3. Pourquoi la méthode `signer()` utilise-t-elle la clé privée ?
4. Pourquoi pouvons-nous partager l'adresse du portefeuille ?
5. Pourquoi ne faut-il pas afficher ou transmettre la clé privée ?

# Partie III — Signer et valider des transactions

## Exercice 5 — Créer une transaction signée

Dans le TP4, nous avons créé une classe `Transaction` contenant un expéditeur, un destinataire et un montant.

Nous allons maintenant ajouter une signature numérique.

### Étape 1 : créer le fichier

Créez un fichier nommé :

`exercice5.py`

Reprenez la classe `Portefeuille` de l'exercice 4.

Ajoutez les imports suivants :

```
import json

from cryptography.exceptions import InvalidSignature
```

### Étape 2 : créer la classe Transaction

```
class Transaction:

    def __init__(self, expediteur, destinataire, montant):

        self.expediteur = expediteur
        self.destinataire = destinataire
        self.montant = montant

        self.signature = None

    def obtenir_message(self):

        donnees = {
            "expediteur": self.expediteur,
            "destinataire": self.destinataire,
            "montant": self.montant
        }

        return json.dumps(
            donnees,
            sort_keys=True,
            separators=(",", ":")
        )

    def signer(self, portefeuille):

        if portefeuille.obtenir_adresse() != self.expediteur:
            raise ValueError("Portefeuille non autorise")

        self.signature = portefeuille.signer(
            self.obtenir_message()
```

```
        )
```

### Étape 3 : créer une transaction

Ajoutez :

```
alice = Portefeuille()
bob = Portefeuille()

transaction = Transaction(
    alice.obtenir_adresse(),
    bob.obtenir_adresse(),
    20
)

transaction.signer(alice)

print("Expediteur :", transaction.expediteur)
print("Destinataire :", transaction.destinataire)
print("Montant :", transaction.montant)
print("Signature :", transaction.signature.hex())
```

### Travail demandé

1. Exécutez le programme.
2. Créez un portefeuille pour Charlie.
3. Créez une transaction de Bob vers Charlie.
4. Signez cette transaction avec le portefeuille de Bob.
5. Essayez de signer une transaction d'Alice avec le portefeuille de Bob.
6. Observez le comportement du programme.

### Questions

1. Pourquoi une transaction doit-elle être signée ?
2. Pourquoi le message signé contient-il l'expéditeur, le destinataire et le montant ?
3. Pourquoi utilisons-nous `json.dumps()` ?
4. Quel est le rôle de la méthode `signer()` de la classe `Transaction` ?
5. Pourquoi vérifions-nous que l'adresse du portefeuille correspond à celle de l'expéditeur ?

## Exercice 6 — Vérifier une transaction signée

Nous allons maintenant compléter la classe `Transaction` pour vérifier automatiquement sa signature.

### Étape 1 : ajouter la méthode de vérification

Dans la classe `Transaction`, ajoutez :

```
    def verifier_signature(self):
```

```
        if self.signature is None:
            return False

        try:

            cle_bytes = bytes.fromhex(self.expediteur)

            cle_publique = Ed25519PublicKey.from_public_bytes(
                cle_bytes
            )

            cle_publique.verify(
                self.signature,
                self.obtenir_message().encode("utf-8")
            )

            return True

        except (InvalidSignature, ValueError):
            return False
```

Ajoutez également cet import au début du fichier :

```
from cryptography.hazmat.primitives.asymmetric.ed25519 import (
    Ed25519PublicKey
)
```

## Étape 2 : tester une transaction valide

Ajoutez :

```
alice = Portefeuille()
bob = Portefeuille()

transaction = Transaction(
    alice.obtenir_adresse(),
    bob.obtenir_adresse(),
    20
)

transaction.signer(alice)

print(
    "Signature valide :",
    transaction.verifier_signature()
)
```

## Étape 3 : modifier la transaction

Ajoutez :

```
transaction.montant = 200

print(
    "Signature valide apres modification :",
    transaction.verifier_signature()
)
```

### Travail demandé

1. Exécutez le programme.
2. Vérifiez la signature avant modification.
3. Modifiez le montant de la transaction.
4. Vérifiez à nouveau la signature.
5. Recommencez en modifiant le destinataire.
6. Testez une transaction qui ne possède aucune signature.

### Questions

1. Pourquoi la méthode `verifier_signature()` utilise-t-elle la clé publique ?
2. Pourquoi la signature est-elle vérifiée sur le contenu de la transaction ?
3. Que se passe-t-il si le montant est modifié après la signature ?
4. Pourquoi une transaction sans signature doit-elle être refusée dans notre modèle ?
5. Une signature valide suffit-elle à garantir qu'une transaction peut être exécutée ? Justifiez.

# Partie IV — Application : validation complète d'une transaction

## Exercice 7 — Combiner signature et vérification du solde

Dans le TP4, nous avons développé une fonction de validation fondée sur les soldes.

Dans ce TP, nous avons ajouté les signatures numériques.

Nous allons maintenant combiner ces deux mécanismes.

### Étape 1 : préparer le programme

Créez un fichier nommé :

`exercice7.py`

Reprenez les classes `Portefeuille` et `Transaction` développées précédemment, avec leurs méthodes de signature et de vérification.

### Étape 2 : créer les portefeuilles

```
alice = Portefeuille()
bob = Portefeuille()
charlie = Portefeuille()
```

### Étape 3 : initialiser les soldes

```
soldes = {
    alice.obtenir_adresse(): 100,
    bob.obtenir_adresse(): 50,
    charlie.obtenir_adresse(): 30
```

```
}
```

## Étape 4 : créer la fonction de validation

```
def valider_transaction(transaction, soldes):

    if not transaction.verifier_signature():
        return False

    if transaction.expediteur not in soldes:
        return False

    if transaction.destinataire not in soldes:
        return False

    if type(transaction.montant) is not int:
        return False

    if transaction.montant <= 0:
        return False

    if soldes[transaction.expediteur] < transaction.montant:
        return False

    return True
```

Dans cette version, les montants doivent être des entiers positifs afin d'éviter les problèmes de précision liés aux nombres à virgule flottante.

## Étape 5 : exécuter une transaction

Ajoutez :

```
def executer_transaction(transaction, soldes):

    if not valider_transaction(transaction, soldes):
        return False

    soldes[transaction.expediteur] -= transaction.montant

    soldes[transaction.destinataire] += transaction.montant

    return True
```

## Étape 6 : tester

```
transaction1 = Transaction(
    alice.obtenir_adresse(),
    bob.obtenir_adresse(),
    20
)

transaction1.signer(alice)

print(executer_transaction(transaction1, soldes))

print(soldes)
```

### Travail demandé

1. Exécutez le programme.
2. Créez une transaction de Bob vers Charlie.
3. Signez-la avec le portefeuille de Bob.
4. Vérifiez sa validité.
5. Exécutez la transaction.
6. Créez une transaction dont le montant dépasse le solde de l'expéditeur.
7. Créez une transaction et modifiez son montant après sa signature.
8. Essayez d'exécuter une transaction sans signature.

### Questions

1. Pourquoi devons-nous vérifier la signature et le solde ?
2. Quelle vérification permet de détecter une modification de la transaction ?
3. Quelle vérification permet de détecter un montant supérieur au solde ?
4. Pourquoi la validation doit-elle précéder la modification des soldes ?
5. Quels autres contrôles faudrait-il ajouter pour construire un système de paiement plus réaliste ?

# Partie V — Étude de cas : tentative d'usurpation d'identité

## Exercice 8 — Peut-on envoyer de l'argent à la place d'un autre utilisateur ?

Considérons trois utilisateurs :

- Alice possède 100 unités.
- Bob possède 50 unités.
- Charlie possède 30 unités.

Chaque utilisateur possède son propre portefeuille cryptographique.

Bob souhaite transférer 80 unités depuis le compte d'Alice vers son propre compte, sans l'autorisation d'Alice.

### Travail demandé

1. Créez les trois portefeuilles.
2. Initialisez les soldes.
3. Créez une transaction indiquant Alice comme expéditeur et Bob comme destinataire.

4. Essayez de signer cette transaction avec le portefeuille de Bob.
5. Observez le comportement du programme.
6. Créez ensuite une transaction correctement signée par Alice.
7. Comparez les deux situations.

### Questions

1. Pourquoi Bob ne devrait-il pas pouvoir signer une transaction à la place d'Alice ?
2. Quel est le rôle de la clé privée dans cette protection ?
3. Pourquoi la clé publique ne permet-elle pas de produire une signature valide ?
4. Que se passerait-il si Bob obtenait la clé privée d'Alice ?
5. Pourquoi la protection des clés privées est-elle essentielle dans une blockchain ?

# Partie VI — Questions de compréhension

Répondez individuellement aux questions suivantes.

1. Qu'est-ce qu'un portefeuille cryptographique ?
2. Quelle est la différence entre une clé publique et une clé privée ?
3. Pourquoi parle-t-on de cryptographie asymétrique ?
4. Quel est le rôle d'une signature numérique ?
5. Quelle clé permet de signer une transaction ?
6. Quelle clé permet de vérifier une signature ?
7. Pourquoi une transaction doit-elle être signée avant d'être envoyée au réseau ?
8. Comment peut-on détecter qu'une transaction a été modifiée après sa signature ?
9. Une signature numérique est-elle équivalente à un hash ? Justifiez.
10. Pourquoi la clé privée doit-elle rester secrète ?
11. Une signature valide garantit-elle que l'expéditeur possède suffisamment de fonds ?
12. Pourquoi faut-il vérifier à la fois la signature et le solde ?
13. Quelle est la différence entre une adresse de portefeuille et une clé privée ?
14. Pourquoi notre programme reste-t-il une simulation simplifiée d'un système blockchain ?
15. Quels risques apparaissent lorsqu'un utilisateur perd ou divulgue sa clé privée ?

# Partie VII — Synthèse

Complétez le tableau suivant.

| Notion | Définition ou explication | Exemple |
|---|---|---|

| | | |
|---|---|---|
| Portefeuille cryptographique | | |
| Clé privée | | |
| Clé publique | | |
| Cryptographie asymétrique | | |
| Signature numérique | | |
| Signature valide | | |
| Vérification de signature | | |
| Adresse de portefeuille | | |
| Authentification cryptographique | | |
| Intégrité d'une transaction | | |
| Validation d'une transaction | | |
| Protection des clés | | |

# Travail à rendre

Chaque étudiant doit remettre :

1. Les fichiers Python des exercices réalisés.
2. Le programme permettant de générer une paire de clés cryptographiques.
3. Le programme permettant de signer et de vérifier un message.
4. La classe `Portefeuille`.
5. La classe `Transaction` avec les méthodes de signature et de vérification.
6. Le programme combinant vérification de signature et contrôle des soldes.
7. Les réponses aux questions des exercices et de compréhension.